



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPT. OF SOFTWARE TECHNOLOGY AND METHODOLOGY

Intrusion Detection System (IDS) using Machine Learning

Supervisor:

Nándor Kelecsényi

PhD student

Author:

Serag Rahmi

Computer Science BSc

Budapest, 2026

Contents

1	Introduction	5
1.1	Background	5
1.2	Motivation	5
1.3	Problem Statement	6
1.4	Aim of the Thesis	7
1.5	Research Approach	7
1.6	Main Contributions	7
1.7	Structure of the Thesis	8
2	Background and Related Concepts	9
2.1	Introduction	9
2.2	Intrusion Detection Systems	9
2.3	Types of Intrusion Detection Systems	9
2.4	Detection Approaches	10
2.5	Machine Learning in Intrusion Detection	10
2.6	Flow-Based Traffic Analysis	11
2.7	Feature Consistency	11
2.8	Machine Learning Models Used	11
2.9	Public Datasets in IDS Research	12
2.10	Evaluation Metrics	12
2.11	Virtual Lab Environment	13
2.12	Summary	13
3	Preliminary Approach and Design Changes	14
3.1	Introduction	14
3.2	Initial Project Idea	14
3.3	Why Anomaly Detection Was Considered	15
3.4	Tools Used in the Initial Approach	15

3.5	Workflow of the First System	16
3.6	Main Problems of the Initial Approach	16
3.7	Lessons Learned	17
3.8	Why the Project Direction Was Changed	17
3.9	Transition to the Final System	17
3.10	Role of the Public Dataset	18
3.11	Summary	18
4	Final System Design	19
4.1	Introduction	19
4.2	General Architecture	19
4.3	Virtual Lab Environment	21
4.4	Traffic Capture and File Transfer	22
4.5	Feature Extraction with CICFlowMeter	22
4.6	Machine Learning Model Integration	23
4.7	Live Detection Logic	23
4.8	Main Python Components	23
4.9	Folder Structure of the Final System	24
4.10	Docker-Based Execution	25
4.11	Benefits and Limitations of the Final Design	27
4.12	Summary	27
5	Dataset Preparation and Preprocessing	28
5.1	Introduction	28
5.2	Choice of Dataset	28
5.3	Files Used from CICIDS2017	29
5.4	Column Cleaning and Alignment	29
5.5	Label Preparation	29
5.6	Feature Cleaning	30
5.7	Final Dataset	30
5.8	Connection Between Offline and Live Data	31
5.9	Implementation Notes	31
5.10	Discussion	33
5.11	Summary	34

6	Model Training and Evaluation	35
6.1	Introduction	35
6.2	Training Goal	35
6.3	Prepared Dataset Used for Training	36
6.4	Sampling, Splitting, and Scaling	36
6.5	Class Imbalance Handling	36
6.6	Trained Models	37
6.7	Evaluation Metrics	37
6.8	Training Results	37
6.9	Interpretation of the Results	38
6.10	Why Random Forest Was Selected	39
6.11	Saved Training Outputs	39
6.12	Notes About Practical Implementation	39
6.13	Possible Improvements	42
6.14	Summary	42
7	Live Implementation and Validation	43
7.1	Introduction	43
7.2	Live Pipeline	43
7.3	Validation Strategy	46
7.4	Validation on Public CICIDS2017 CSV Files	47
7.5	Validation of the Full PCAP Pipeline	47
7.6	Live Testing in the Virtual Lab	47
7.7	Custom Attack Traffic Generation	48
7.7.1	Port Scanning Traffic	49
7.7.2	Denial-of-Service Traffic	49
7.7.3	Failed Access Traffic	50
7.8	Improved Model with Custom Data	50
7.9	Live Result of the Improved Model	51
7.10	Limitations	52
7.11	Summary	53
8	Conclusion and Future Work	54
8.1	Introduction	54
8.2	Summary of the Thesis	54

8.3	Main Results	55
8.4	Research Questions Revisited	55
8.5	Main Lessons Learned	56
8.6	Strengths and Limitations	56
8.7	Future Work	57
8.8	Final Remarks	57
	Acknowledgements	58
	Bibliography	58
	List of Figures	62
	List of Tables	63
	List of Codes	64

Chapter 1

Introduction

1.1 Background

Computer networks are used in many parts of daily life, such as universities, companies, banks, hospitals, and personal communication. Because of this, network security is an important topic. If a network is attacked, the result can be data loss, service interruption, unauthorized access, or damage to important systems.

One common security tool is the intrusion detection system, usually called IDS. An IDS monitors activity and tries to identify suspicious or malicious behavior [1, 2]. Traditional IDS solutions often use signatures or predefined rules. These methods work well for known attacks, but they can be weaker when the attack pattern is new or not already stored in the rule set.

Machine learning has therefore become a common research direction in intrusion detection [3, 4]. Instead of only comparing traffic with fixed rules, a machine learning model can learn patterns from data and use them to classify new traffic.

This thesis focuses on a machine learning based IDS for network traffic. The system uses flow-based features instead of raw packet payloads. The main idea is to train and compare several machine learning models on a public dataset, then use the best model in a live detection pipeline.

1.2 Motivation

The motivation of this thesis was to build a practical IDS prototype, not only to train a model and report its accuracy. The aim was to connect the selected model to

a live traffic capture process, so that newly captured PCAP files could be converted and classified automatically.

Another motivation was to understand the practical difficulties of using machine learning in a real or semi-real network environment. At first, the process seemed simple: capture traffic, convert it into data, train a model, and detect attacks. In practice, this was harder. Feature mismatch, unstable CSV conversion, and differences between offline and live traffic created several problems. This is also a known issue in machine learning based intrusion detection, because traffic from one environment may differ from the traffic used during training [5].

For this reason, the project developed in two stages. First, an anomaly detection approach was tested. Later, the system was redesigned as a supervised, flow-based IDS using CICIDS2017 and a more stable live pipeline.

1.3 Problem Statement

The main problem of this thesis is how to design and implement a machine learning based intrusion detection system that can classify network traffic as normal or attack in a reliable way.

At the beginning of the work, an anomaly detection approach was considered. The idea was to train a model only on normal traffic and classify different behavior as suspicious. In that version, traffic was captured on Ubuntu, transferred to Windows through Google Drive, and converted from PCAP to CSV using TShark. TShark is a command-line network protocol analyzer that can capture live traffic or read saved capture files [6]. TimesNet was selected because it supports time series tasks, including anomaly detection [7].

This first approach did not work well in practice. The CSV files produced by the early conversion process did not provide a suitable flow-based representation for the model. There was also a mismatch between the training data and the test data. Because of this, the model could not detect attacks reliably, and the workflow was not practical for near real-time use.

The system was therefore redesigned. The final approach uses supervised learning with a public labelled dataset. This made training, testing, and live validation more stable and easier to evaluate.

1.4 Aim of the Thesis

The aim of this thesis is to design and implement a machine learning based intrusion detection system that can distinguish between normal and malicious network traffic.

The main tasks were to study IDS and machine learning based detection, test an initial anomaly detection idea, prepare a public dataset, train and compare several models, select the best model, integrate it into a live pipeline, and validate the system with both public files and live captured traffic.

1.5 Research Approach

The final system uses CICIDS2017 as the main dataset for training and testing. The dataset contains benign traffic and several attack types [8, 9]. In this thesis, the task was simplified into binary classification: BENIGN traffic was labelled as normal, and all other labels were grouped into the attack class.

Four models were trained and compared: Decision Tree, Random Forest, Support Vector Machine, and Neural Network. After the comparison, Random Forest was selected for the live system.

The live environment used three main machines. Ubuntu was the monitored machine where traffic was captured. Kali Linux was used in the virtual lab to generate controlled abnormal traffic. Windows was used as the processing machine, where PCAP files were converted into CSV files and classified by the trained model. The virtual machines used VirtualBox network modes such as host-only, NAT, and bridged networking during different tests [10].

The final pipeline used CICFlowMeter to convert PCAP files into flow-based CSV data. This was important because the live features had to be close to the feature format used during training [11].

1.6 Main Contributions

The main contributions of the thesis are the investigation of an early TimesNet anomaly detection approach, the analysis of its problems, the preparation of a binary CICIDS2017 dataset, the comparison of four machine learning models, the selection

of Random Forest, and the implementation of a live IDS pipeline using Ubuntu, Kali Linux, Windows, shared folders, and CICFlowMeter.

The thesis also validates the system in more than one way. It uses known CICIDS2017 CSV files, public PCAP files, and live captured traffic from the local virtual lab.

1.7 Structure of the Thesis

Chapter 2 introduces intrusion detection systems, flow-based traffic analysis, machine learning models, and evaluation metrics. Chapter 3 explains the first anomaly detection approach and why it was changed. Chapter 4 presents the final system design. Chapter 5 describes dataset preparation and preprocessing. Chapter 6 presents model training and evaluation. Chapter 7 describes the live implementation and validation. Chapter 8 gives the conclusion and possible future work.

Chapter 2

Background and Related Concepts

2.1 Introduction

This chapter gives the background needed for the thesis. It explains intrusion detection systems, the main detection approaches, flow-based traffic analysis, the machine learning models used in this work, and the evaluation metrics.

2.2 Intrusion Detection Systems

An intrusion detection system, or IDS, is a security system that monitors activity and tries to identify suspicious behavior. In a network-based case, the IDS observes communication between devices and looks for patterns that may indicate an attack [1, 2].

The main goal of an IDS is to detect possible threats early. It does not always stop the attack directly, but it can generate alerts, store logs, and help an administrator understand what is happening in the network.

2.3 Types of Intrusion Detection Systems

IDS solutions can be divided into host-based and network-based systems [1]. A host-based IDS monitors one machine and can inspect logs, file changes, user actions, and system events. A network-based IDS monitors traffic between machines.

This thesis uses a network-based approach. The monitored traffic is captured from the Ubuntu machine, converted into flow-based data, and then classified by a machine learning model.

2.4 Detection Approaches

There are two common approaches in intrusion detection: signature-based detection and anomaly-based detection [1, 4].

Signature-based detection compares observed traffic with known attack patterns. This works well for attacks that are already known, but it can fail when the attack does not match an existing signature.

Anomaly-based detection learns what normal traffic looks like and treats unusual behavior as suspicious [3, 4]. This can be useful for unknown attacks, but it needs a stable and correct definition of normal behavior. At the beginning of this project, anomaly detection was tested, but it was difficult to make reliable because of feature mismatch and live data preparation problems.

The final system uses supervised learning. In supervised learning, the model is trained with labelled examples, where the class of each sample is already known. This was more suitable for the final version because CICIDS2017 provided labelled benign and attack traffic [8, 9].

2.5 Machine Learning in Intrusion Detection

Machine learning is useful in intrusion detection because network traffic can contain patterns that are difficult to describe with simple rules. A model can learn from data and later use these learned patterns to classify new traffic [3].

In a typical machine learning based IDS, traffic is collected, features are extracted, data is prepared, models are trained and evaluated, and the best model is used for detection. The quality of the extracted features is very important. If the features are not suitable, even a strong model may perform badly.

This was one of the main lessons of the project. In the first design, the converted CSV files were not suitable for stable learning. In the final design, CICFlowMeter was used to generate flow-based features that were closer to the public training dataset [11]. Another challenge is that traffic collected in one environment may look

different from the data used during training, which can reduce model performance in live use [5].

2.6 Flow-Based Traffic Analysis

This thesis uses flow-level traffic analysis. Instead of using raw packet payloads directly, packets are grouped into flows, and each flow is described by statistical features [11].

A flow usually represents communication between two endpoints during a period of time. The extracted features can include packet counts, packet length values, flow duration, packet rates, inter-arrival times, flag counts, and active or idle times.

This representation reduces the complexity of packet data. The model does not need to inspect every raw packet, but instead works with summarized traffic behavior. Flow-based analysis also fits CICIDS2017, because the dataset is available in flow-based CSV format [8, 9].

2.7 Feature Consistency

Feature consistency is an important practical issue in machine learning based IDS. The training data and the live data must have the same feature structure, meaning, and order.

If a model is trained on one feature format and later receives a different format, its predictions become unreliable. This was one reason why the first approach did not work well. TShark did not produce a stable flow-based representation that matched the later model input.

In the final system, this problem was reduced by using CICIDS2017 for training and CICFlowMeter for live feature extraction. Since both sides use flow-based features, the gap between offline and live data became smaller [11, 8].

2.8 Machine Learning Models Used

Four machine learning models were trained and compared in this thesis: Decision Tree, Random Forest, Support Vector Machine, and Neural Network. These models

are common in machine learning and are also used in intrusion detection research [3, 4].

A Decision Tree makes decisions step by step based on feature values. It is easy to understand and fast to train, but a single tree can overfit if it is not controlled properly [12, 13].

Random Forest is an ensemble method that combines many decision trees. It is usually more stable than a single tree and often works well on structured tabular data [14]. In this thesis, it achieved the best result.

Support Vector Machine tries to separate classes by finding a decision boundary between them [15]. It is an important classification method, but in this project it was slower and less accurate than the tree-based models.

A Neural Network is built from layers of connected units that transform input data step by step [16, 17]. It can learn complex patterns, but in this work it did not outperform Random Forest.

2.9 Public Datasets in IDS Research

Public datasets are important in IDS research because they make experiments easier to repeat and compare. In this thesis, CICIDS2017 was used as the main dataset because it contains labelled benign and attack traffic, includes several attack categories, and provides flow-based CSV files [8, 9].

This was especially useful after the first anomaly detection attempt failed because of unstable feature extraction. Using CICIDS2017 made the model comparison clearer and more reliable.

Other IDS datasets also exist, such as older KDD-based datasets, UNSW-NB15, CSE-CIC-IDS2018, and CIC-DDoS2019 [18, 19, 20, 21]. However, these datasets may use different feature structures and traffic distributions. For this thesis, CICIDS2017 was kept as the main dataset to keep the scope manageable.

2.10 Evaluation Metrics

Accuracy, precision, recall, F1-score, and confusion matrices were used to evaluate the models [22, 23]. Accuracy shows the ratio of correct predictions, but it can be misleading when the classes are imbalanced. Precision shows how many predicted

attacks were actually attacks. Recall shows how many real attacks were detected. F1-score combines precision and recall, so it was useful for comparing the models. A confusion matrix gives a more detailed view by showing correct and incorrect predictions for each class.

2.11 Virtual Lab Environment

The practical part of the thesis was tested in a virtual lab. Ubuntu was used as the monitored machine, Kali Linux was used to generate controlled abnormal traffic, and Windows was used as the processing machine.

This setup made it possible to test the system without using an external production network. It also connected the offline machine learning part with live traffic capture and detection. The virtual machines used different adapter modes, such as host-only, NAT, and bridged networking, depending on the test scenario [10].

2.12 Summary

This chapter introduced the main concepts needed for the thesis. It explained IDS types, detection approaches, machine learning based detection, flow-based features, public IDS datasets, evaluation metrics, and the virtual lab environment. The next chapter presents the first design idea and explains why the system was redesigned.

Chapter 3

Preliminary Approach and Design Changes

3.1 Introduction

The final system in this thesis was not the first design of the project. Before the final supervised IDS was built, an anomaly detection approach was tested. The first idea was to capture traffic on Ubuntu, transfer the captured files to Windows, convert them into CSV format, and use a deep learning model to detect abnormal behavior.

This chapter explains the first approach, the tools that were used, the main problems, and why the project was redesigned.

3.2 Initial Project Idea

At the beginning of the thesis, the idea was to build a normal-only anomaly detection system. Instead of training the model with both benign and attack traffic, the model would learn only from normal traffic. After that, traffic that looked different from the learned normal pattern would be treated as suspicious.

This idea was attractive because in real networks it is often easier to collect normal traffic than many labelled attack examples. It also seemed useful for detecting unknown attacks, because anomaly detection does not depend only on previously seen malicious samples [3, 4].

The early workflow was simple: traffic was captured on Ubuntu, moved to Windows, converted from PCAP to CSV, and then used for normal-only model training. At this stage, TShark was used for conversion and TimesNet was selected as the anomaly detection model.

3.3 Why Anomaly Detection Was Considered

Anomaly detection matched the first idea of the project. The live lab environment was expected to produce mostly normal traffic at the beginning, so it seemed reasonable to train a model on that normal traffic and later test whether unusual traffic could be detected.

However, this approach needed a stable data representation. The model had to receive the same type of features during training and testing. In practice, this became one of the main problems. The data produced during live testing did not match the data used during training well enough. This kind of mismatch is a known difficulty when machine learning models are used outside the same data distribution that they were trained on [5].

3.4 Tools Used in the Initial Approach

The first version used Ubuntu, Windows, Google Drive, TShark, and TimesNet.

Ubuntu was used to capture network traffic and save it as PCAP files. Windows was used for data preparation and model execution. Google Drive was first used to move the captured files from Ubuntu to Windows. This worked for simple file transfer, but it was too slow and inconvenient for a workflow that was meant to become close to live detection.

TShark was used to convert PCAP files into CSV format. Since TShark is part of the Wireshark ecosystem, it looked like a reasonable choice for extracting packet information [6]. However, the exported CSV data was closer to packet-level information and did not provide the stable flow-based feature structure needed for the final IDS.

TimesNet was selected as the first model because it supports time series tasks, including anomaly detection [7]. The plan was to let the model learn normal traffic

patterns and detect traffic that looked different. This idea was interesting, but it depended strongly on the quality and consistency of the input data.

3.5 Workflow of the First System

The first system followed this workflow:

1. capture traffic on Ubuntu,
2. transfer the PCAP files to Windows using Google Drive,
3. convert the PCAP files into CSV using TShark,
4. train a TimesNet based anomaly detection model on normal traffic,
5. test whether different traffic could be detected as attack.

At first, this workflow looked possible. After testing, it became clear that the main problems were not only in the model, but also in the data preparation and transfer process.

3.6 Main Problems of the Initial Approach

The first problem was feature mismatch. The model needed a stable input format, but the CSV files produced during training and testing did not have a reliable matching structure. Because of this, the model was not learning and testing on fully comparable data.

The second problem was the CSV representation itself. The TShark output was not suitable enough for this IDS task because it did not create the same kind of flow-based statistical features that were later needed. For this thesis, flow-level features such as duration, packet counts, inter-arrival times, and packet length statistics were more useful than raw packet-style exports.

The third problem was the transfer method. Moving files through Google Drive added delay and manual steps. This made the first design unsuitable for near real-time or repeated live detection.

Because of these issues, the TimesNet based system did not produce reliable detection results. The main reason was not only the model choice, but the fact that the input data and the pipeline were not stable enough.

3.7 Lessons Learned

The first attempt was still useful because it showed what had to be improved. The most important lesson was that feature extraction is critical in a machine learning based IDS. If the extracted features are not stable, the whole system becomes unreliable.

Another lesson was that live detection needs a direct workflow. A slow transfer process with extra manual steps makes the system harder to use. It also became clear that the training data and live data must be as similar as possible. It is not enough to have a CSV file; the features must have the same meaning and structure.

This first attempt also showed that anomaly detection may be useful in theory, but it was not the most reliable choice for the first working version of this project.

3.8 Why the Project Direction Was Changed

After these problems, the project was redesigned. The first major change was to use a public labelled dataset instead of relying only on normal traffic. This made supervised learning possible and allowed the models to be compared more clearly.

The second change was replacing TShark with CICFlowMeter for the main feature extraction. CICFlowMeter was more suitable because it generates bidirectional flow records and statistical traffic features from PCAP files [11]. This made the live data closer to the public dataset format.

The third change was replacing Google Drive with a shared folder between Ubuntu and Windows. This made the file transfer faster and more practical for repeated live processing.

Together, these changes moved the project from an unstable anomaly detection attempt to a more practical supervised IDS system.

3.9 Transition to the Final System

After the redesign, CICIDS2017 was used as the main dataset for training and evaluation. It contains benign traffic and several attack scenarios, and it is commonly used in intrusion detection research [8, 9]. The task was simplified into binary clas-

sification, where BENIGN traffic was treated as normal and all other labels were treated as attack.

Four models were then trained and compared: Decision Tree, Random Forest, Support Vector Machine, and Neural Network. Random Forest gave the best result and was selected for the live detection pipeline.

The final live workflow also became cleaner. Ubuntu captured the traffic, the PCAP files were transferred through a shared folder, CICFlowMeter converted them into flow-based CSV files, and the trained model classified the flows as normal or attack.

3.10 Role of the Public Dataset

The use of a public dataset was one of the key changes in the project. CICIDS2017 gave the project a reliable labelled dataset and a flow-based structure that matched the final pipeline better than the earlier TShark export [8, 9].

Using a public dataset made it possible to prepare the data in a controlled way, compare several models, use standard evaluation metrics, and connect the offline experiments to the live IDS pipeline.

3.11 Summary

This chapter described the first design idea and why it was changed. The early system used anomaly detection, normal-only training, TimesNet, TShark, and Google Drive based transfer. The main problems were feature mismatch, unsuitable CSV structure, and an impractical live workflow.

These problems led to the final design, which used supervised learning, CICIDS2017, CICFlowMeter, shared folders, and a live PCAP-to-CSV-to-model pipeline. The next chapter presents the architecture of this final system.

Chapter 4

Final System Design

4.1 Introduction

After the first approach was found to be unreliable, the system was redesigned. The final version of the project used a supervised machine learning approach and a more stable live processing pipeline.

The main goal of the final design was to connect offline model training with live traffic testing. Public labelled data was used for training, while traffic captured in the virtual lab was used for validation. The system was built around flow-based feature extraction, because the live data had to match the training data as much as possible.

This chapter presents the final architecture, the role of each machine, the main tools, and the data flow from packet capture to final detection.

4.2 General Architecture

The final IDS system used three machines. Ubuntu was the monitored machine and captured network traffic. Kali Linux was used to generate controlled abnormal traffic during live tests. Windows was the main processing machine, where conversion, machine learning prediction, and result storage were done.

The workflow of the system was:

1. capture traffic on Ubuntu,
2. save the captured traffic as PCAP files,

3. transfer the PCAP files to Windows through a shared folder,
4. convert the PCAP files into flow-based CSV files with CICFlowMeter,
5. classify the flows with the trained machine learning model,
6. produce a final decision for each file as NORMAL or ATTACK.

Figure 4.1 shows the main architecture of the final IDS system.

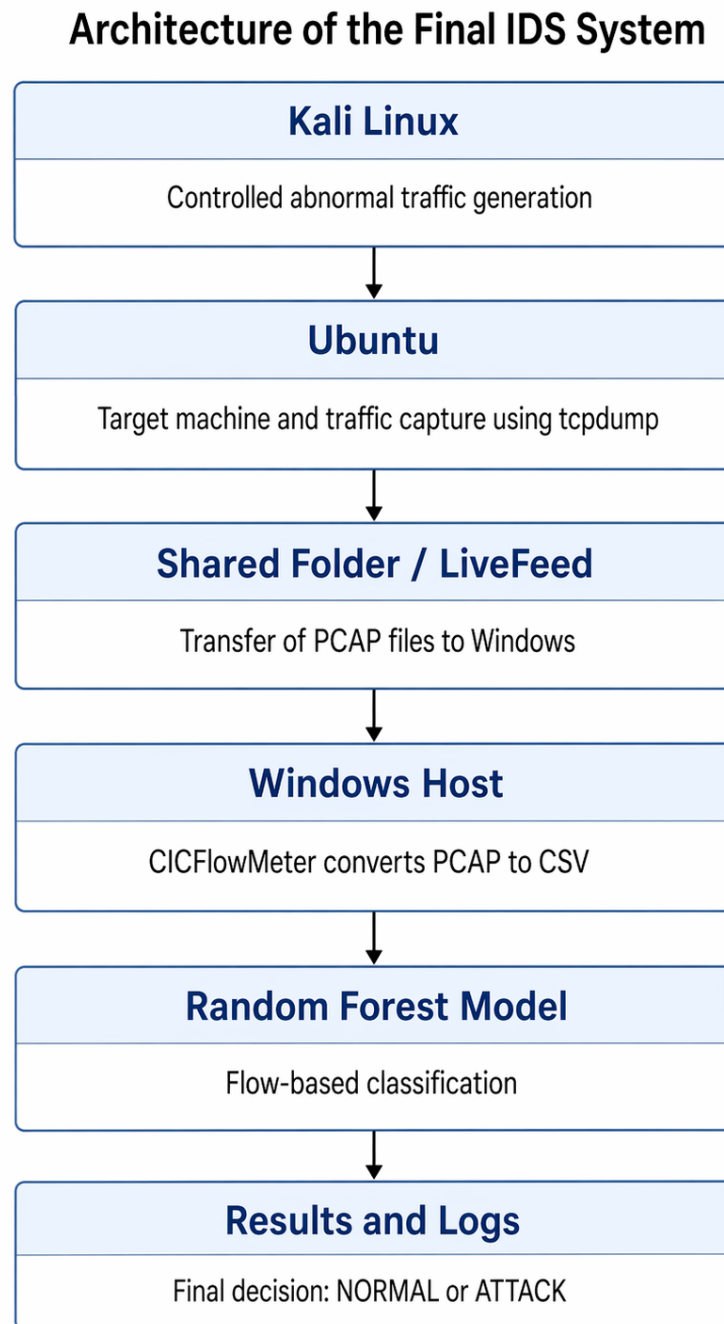


Figure 4.1: Architecture of the final IDS system.

This design was more practical than the first version because it removed the Google Drive transfer step and replaced TShark with CICFlowMeter for flow-based feature extraction.

4.3 Virtual Lab Environment

The practical validation was done in a virtual lab created with VirtualBox. This allowed the system to be tested in a controlled environment without using an external production network [10].

Figure 4.2 shows the virtual lab network used during the live tests.

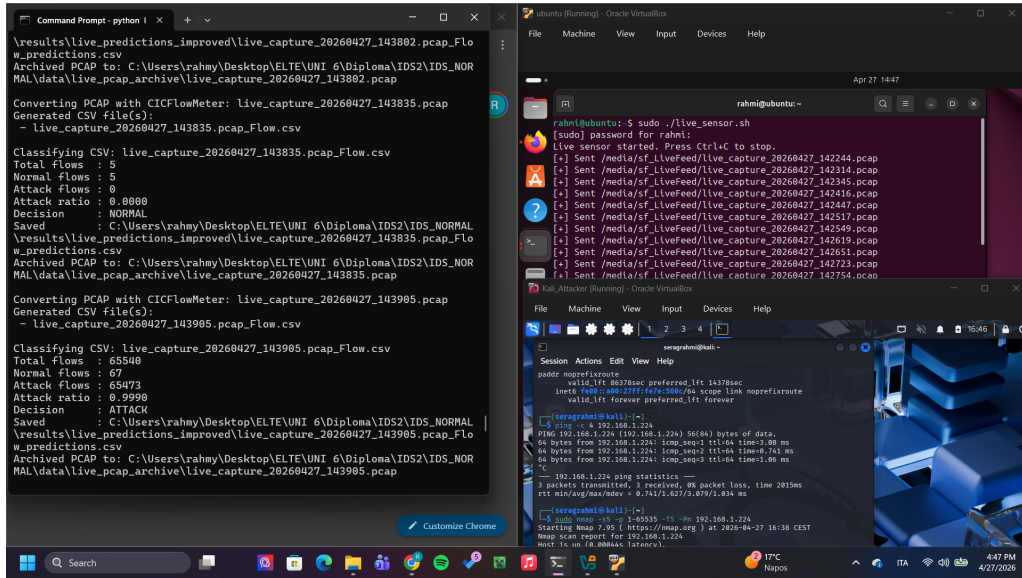


Figure 4.2: Virtual lab network used for live IDS testing.

Ubuntu and Kali Linux were used as virtual machines. Ubuntu captured the traffic, while Kali was used only for controlled test traffic during validation. Windows acted as the host and processing machine. This separation made the system easier to manage: Ubuntu focused on packet capture, Kali produced test traffic, and Windows handled conversion and prediction.

The virtual lab was useful because the machines could be restarted and configured again when needed. It also reduced the risk of affecting any real external system.

4.4 Traffic Capture and File Transfer

Traffic was captured on Ubuntu using `tcpdump`. The captured traffic was saved in PCAP format, which is a standard format for packet captures [24].

The capture process used fixed time windows. Short windows such as 10 seconds were tested first, but 30-second windows were more useful because very short captures sometimes produced too few flows. Each capture was saved with a timestamped file name, which avoided overwriting older files and made it easier for the Windows pipeline to detect new inputs.

The PCAP files were saved into a VirtualBox shared folder that was visible from Windows. This replaced the earlier Google Drive based transfer method. The shared folder was faster, simpler, and more suitable for repeated live processing.

4.5 Feature Extraction with CICFlowMeter

CICFlowMeter was used to convert PCAP files into CSV files with flow-based features. This was an important design choice because CICFlowMeter produces features that are close to the structure used in the CICIDS2017 dataset [11].

Figure 4.3 shows the CICFlowMeter tool used for PCAP-to-CSV conversion.

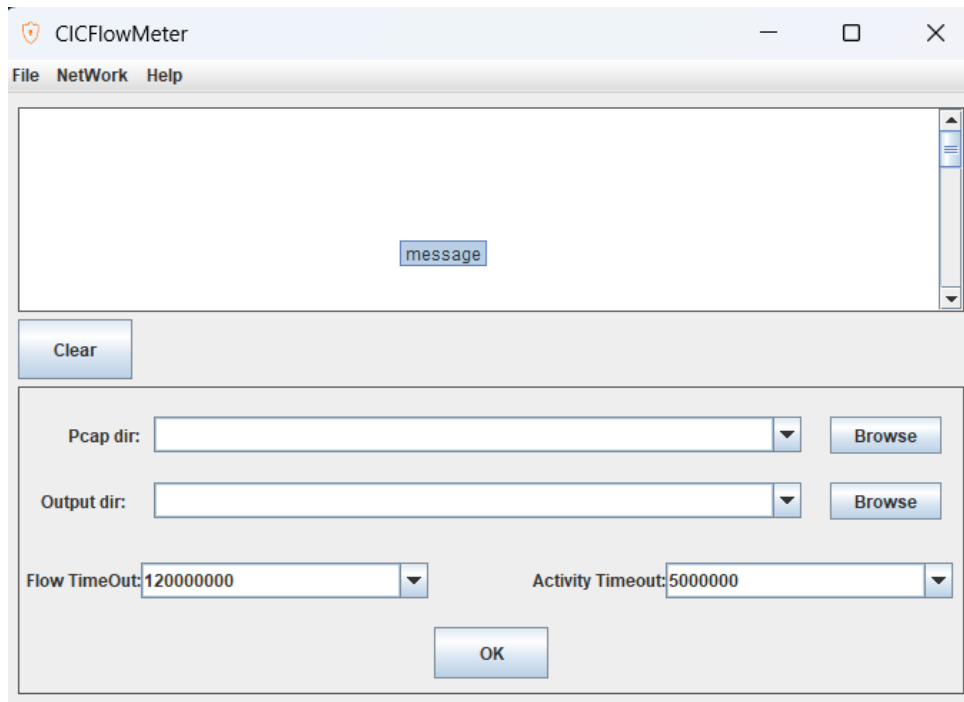


Figure 4.3: CICFlowMeter interface used for PCAP-to-CSV conversion.

The input of CICFlowMeter was a folder containing PCAP files. The output was a CSV file where each row represented one network flow. The generated features included packet counts, packet lengths, flow duration, byte rates, inter-arrival time values, flag counters, and forward/backward traffic statistics.

This step was critical because the live system depended on feature consistency. If the live CSV files had a very different structure from the training data, the trained model would not be reliable.

4.6 Machine Learning Model Integration

The machine learning models were trained offline on the prepared CICIDS2017 dataset. After model comparison, Random Forest was selected as the final model because it achieved the best overall performance. Random Forest combines several decision trees and uses their combined prediction for classification [14].

The live system did not train the model again. It only loaded the saved model and used it for prediction on new CSV files. The live prediction process also required the saved scaler and the saved feature column list, so that the live data could be prepared in the same way as the training data.

4.7 Live Detection Logic

After a PCAP file was converted into CSV format, each row was treated as one flow. The trained model predicted whether each flow was normal or attack.

The script then calculated the total number of flows, the number of predicted attack flows, and the attack ratio. Based on these values, it produced one final decision for the whole file. This two-level logic was useful because a few suspicious flows can appear even in mostly normal traffic. The file-level decision helped reduce false alarms.

4.8 Main Python Components

The final system used several Python scripts. The dataset preparation script cleaned and merged CICIDS2017 data and created the binary dataset. The training script trained and compared the machine learning models. The CSV detector was

used to test already existing CSV files. The live pipeline script watched the PCAP folder, called CICFlowMeter, prepared the generated CSV file, loaded the trained model, and produced the final prediction.

This structure made the project easier to test because each part had a clear role.

4.9 Folder Structure of the Final System

The project was organized into separate folders for raw data, processed data, custom data, models, results, logs, and scripts. This made debugging easier and helped separate offline training files from live captured files.

Code 4.1 shows the main folder structure used in the project.

```
1 IDS2/
2 |-- LiveFeed/
3 |   |-- live_capture_*.pcap
4 |
5 |-- LiveFeed_CSV/
6 |   |-- live_capture_*.pcap_Flow.csv
7 |
8 |-- CICFlowMeter-master/
9 |   |-- build/
10 |     |-- distributions/
11 |       |-- CICFlowMeter-4.0/
12 |         |-- bin/
13 |           |-- cfm.bat
14 |
15 |-- IDS_NORMAL/
16 |   |-- data/
17 |     |-- raw_cicids2017/
18 |     |-- processed/
19 |     |-- live_pcap_archive/
20 |     |-- live_pcap_failed/
21 |
22 |   |-- custom_data/
23 |     |-- csv/
24 |     |-- processed/
```

```
25 | |
26 | |-- models/
27 | |   |-- random_forest.pkl
28 | |   |-- random_forest_improved.pkl
29 | |   |-- scaler.pkl
30 | |   |-- scaler_improved.pkl
31 | |
32 | |-- results/
33 | |   |-- live_predictions/
34 | |   |-- live_predictions_improved/
35 | |   |-- model_comparison_results.csv
36 | |
37 | |-- logs/
38 | |
39 | |-- prepare_dataset.py
40 | |-- prepare_custom_dataset.py
41 | |-- train_ids.py
42 | |-- train_ids_improved.py
43 | |-- live_detector.py
44 | |-- live_pipeline_improved.py
45 | |-- Dockerfile
46 | |-- requirements.txt
```

Code 4.1: Main folder structure of the IDS project

The `LiveFeed`, `LiveFeed_CSV`, and `CICFlowMeter-master` folders were placed outside the main `IDS_NORMAL` project folder. The `LiveFeed` folder stored incoming PCAP files, `LiveFeed_CSV` stored the CSV files generated by `CICFlowMeter`, and `CICFlowMeter-master` contained the `CICFlowMeter` executable used for PCAP-to-CSV conversion. The `IDS_NORMAL` folder contained the machine learning code, models, results, logs, and processed datasets.

4.10 Docker-Based Execution

Docker was used to make the Python machine learning environment easier to reproduce. In this project, Docker was mainly used for the improved model training

script. This allowed the training process to run inside an isolated Python environment instead of depending only on the Python packages installed directly on Windows.

The Docker image was created with a simple `Dockerfile`. Code 4.2 shows the Docker configuration used in the project.

```
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt /tmp/requirements.txt
6
7 RUN pip install --no-cache-dir -r /tmp/requirements.txt
8
9 CMD ["python"]
```

Code 4.2: Dockerfile used for the Python machine learning environment

The image installed the required Python libraries from `requirements.txt`. The project folder was mounted into the container, so the container could read the datasets and save the trained model, scaler, and result files back into the original project directory.

Code 4.3 shows the main Docker commands used to build the image and run the improved training script.

```
1 docker build -t ids-ml-system .
2
3 docker run --rm -it -v "%cd%:/app" ids-ml-system python
   train_ids_improved.py
```

Code 4.3: Docker commands used to build and run the IDS training environment

Only the Python machine learning part was containerized. The full live capture pipeline was kept outside Docker because it depended on VirtualBox network interfaces, shared folders, CICFlowMeter, Java, and native packet libraries. This was a practical decision, because the training part could still be reproduced with Docker while the live capture and conversion steps remained on the host machine.

4.11 Benefits and Limitations of the Final Design

The final design had several advantages over the first approach. It used a public labelled dataset, flow-based feature extraction, direct shared folder transfer, and a practical live pipeline from PCAP capture to final classification. It also allowed both offline testing and live validation in the virtual lab.

However, the system still had limitations. The live pipeline worked in repeated file-based windows, not as a fully continuous high-speed IDS. The live accuracy also depended on how similar the lab traffic was to the training data. Finally, the output was binary, so it only reported NORMAL or ATTACK and did not identify the exact attack type.

4.12 Summary

This chapter presented the final system design. The system used Ubuntu for traffic capture, Kali Linux for controlled abnormal traffic, Windows for processing, CICFlowMeter for flow extraction, and Random Forest for classification. The design solved the main problems of the first approach and created a stable workflow for both dataset-based experiments and live validation. The next chapter describes the dataset preparation and preprocessing steps.

Chapter 5

Dataset Preparation and Preprocessing

5.1 Introduction

This chapter explains the dataset used in the final system and the preprocessing steps applied before model training. Since the project is based on machine learning, the quality and consistency of the input data were very important.

The final system used CICIDS2017 as the main public dataset. CICIDS2017 contains benign traffic and several attack scenarios, and it is commonly used in intrusion detection research [8, 9]. The raw CSV files could not be used directly. They first had to be merged, cleaned, converted into binary labels, and aligned with the live CICFlowMeter output format.

5.2 Choice of Dataset

CICIDS2017 was selected because it provides labelled traffic, includes both normal and malicious behaviour, and is available in flow-based CSV format [8, 9]. This matched the final design of the thesis, where PCAP files are converted into flow records before classification.

The choice of CICIDS2017 was also connected to the problems in the first approach. The earlier live-only anomaly detection idea suffered from unstable feature extraction and feature mismatch. A public dataset with a clear structure made the

training and evaluation process more reliable. Public datasets are also useful because they make experiments easier to repeat and compare with other work [25, 4].

5.3 Files Used from CICIDS2017

Eight CICIDS2017 CSV files were used in this project. They covered different days and attack scenarios: Monday-WorkingHours, Tuesday-WorkingHours, Wednesday-workingHours, Thursday WebAttacks, Thursday Infiltration, Friday Morning, Friday PortScan, and Friday DDoS.

These files were copied into the raw dataset folder and processed with a Python script. One file, `Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv`, required fallback encoding because it could not be read correctly with UTF-8 only. For this reason, the script was written to try more than one encoding.

After all eight files were read and merged, the initial dataset contained 3,119,345 rows and 85 columns.

5.4 Column Cleaning and Alignment

After loading the data, the column names were cleaned by removing unnecessary spaces. Some columns were also renamed to make the offline dataset closer to the live CSV files produced later by CICFlowMeter. For example, `Source IP` was renamed to `Src IP`, `Source Port` to `Src Port`, `Destination IP` to `Dst IP`, and `Destination Port` to `Dst Port`.

This step was important because the live detection pipeline depended on feature consistency between training data and live data. CICFlowMeter was used later to generate bidirectional flow records and statistical features from PCAP files [11].

One duplicate-style feature column, `Fwd Header Length.1`, was also removed. Keeping repeated or duplicate columns could make feature alignment harder during training and live prediction.

5.5 Label Preparation

The original CICIDS2017 files contain several labels. For this thesis, the task was simplified into binary classification. The `BENIGN` label was treated as normal traffic,

and all other labels were grouped into the attack class.

A new column called **Label_Binary** was created. The value 0 represented normal traffic, and the value 1 represented attack traffic. This binary setup was chosen because the main goal of the first working live IDS was to decide whether a traffic batch was normal or suspicious, not to identify the exact attack type.

Before removing invalid and duplicate rows, the binary label distribution was 2,273,097 normal rows and 846,248 attack rows. This showed that the dataset was imbalanced, which is common in intrusion detection datasets [4].

5.6 Feature Cleaning

Some columns were not suitable as machine learning features because they were identifiers or metadata rather than stable traffic behaviour values. For this reason, **Flow ID**, **Src IP**, **Dst IP**, and **Timestamp** were removed.

After that, the remaining feature columns were converted to numeric values. Values that could not be converted were treated as missing values. This was done with Python and pandas, which is commonly used for CSV processing and data preparation [26].

The dataset also contained missing and infinite values. Infinite values were first replaced with missing values, and rows containing invalid values were removed. In total, 291,469 rows were removed because of NaN or infinite values.

Duplicate rows were then removed. This deleted 16,918 rows. Removing duplicates was important because repeated records could bias training and make evaluation results look better than they really were.

5.7 Final Dataset

After preprocessing, the final cleaned dataset contained 2,810,958 rows and 81 columns. From these columns, 79 were usable machine learning features. The remaining two columns were the original **Label** column and the generated **Label_Binary** column.

The final class distribution was 2,261,990 normal rows and 548,968 attack rows. The dataset was still imbalanced, but it was suitable for binary classification. Class

imbalance was handled later during training through class weights and suitable evaluation metrics.

The preprocessing script saved the cleaned binary dataset, the original label summary, the binary label summary, and the final feature column list. These saved files were used later by the training and live detection scripts.

5.8 Connection Between Offline and Live Data

One of the main goals of preprocessing was to make the offline training data and live data as similar as possible. This was important because the final system was not only an offline classifier. The selected model was later used with real captured traffic.

For this reason, preprocessing included column name cleaning, feature renaming toward the live CICFlowMeter format, removal of non-feature identifiers, and saving the final feature column list. These steps helped connect the offline machine learning stage with the live detection pipeline. This was especially important because models trained on one traffic distribution may not always generalize well to another environment [5].

5.9 Implementation Notes

The preprocessing workflow was implemented in Python. The script loaded the selected CICIDS2017 CSV files, handled encoding differences, merged the files, cleaned and renamed columns, created binary labels, removed invalid and duplicate rows, and saved the cleaned output files.

Code 5.1 shows the main preprocessing logic used in the dataset preparation script.

```
1 # Read selected CICIDS2017 CSV files
2 csv_files = sorted(RAW_DIR.glob("*.csv"))
3
4 dataframes = []
5 for file in csv_files:
6     df = read_csv_robust(file)
7     df.columns = df.columns.str.strip()
```



```
8     dataframes.append(df)
9
10 # Merge files and clean column names
11 df_all = pd.concat(dataframes, ignore_index=True)
12 df_all.columns = df_all.columns.str.strip()
13
14 # Rename columns toward live CICFlowMeter style
15 df_all.rename(columns=COLUMN_RENAME_MAP, inplace=True)
16
17 # Remove duplicate-style column if it exists
18 if "Fwd Header Length.1" in df_all.columns:
19     df_all.drop(columns=["Fwd Header Length.1"], inplace=True)
20
21 # Create binary label
22 df_all["Label"] = df_all["Label"].astype(str).str.strip()
23 df_all["Label_Binary"] = (
24     df_all["Label"].str.upper() != "BENIGN"
25 ).astype(int)
26
27 # Drop identifier columns
28 existing_drop_cols = [col for col in DROP_COLUMNS if col in
29     df_all.columns]
30
31 df_all.drop(columns=existing_drop_cols, inplace=True)
32
33 # Convert feature columns to numeric values
34 feature_cols = [
35     col for col in df_all.columns
36     if col not in ["Label", "Label_Binary"]
37 ]
38 df_all[feature_cols] = df_all[feature_cols].apply(
39     pd.to_numeric,
40     errors="coerce"
41 )
42
43 # Remove invalid and duplicate rows
44 df_all.replace([np.inf, -np.inf], np.nan, inplace=True)
```

```
43 df_all.dropna(inplace=True)
44 df_all.drop_duplicates(inplace=True)
45
46 # Save processed dataset and feature list
47 df_all.to_csv(PROCESSED_DIR / "cicids2017_binary.csv", index=
    False)
48
49 feature_list = pd.DataFrame({
50     "feature_name": [
51         col for col in df_all.columns
52         if col not in ["Label", "Label_Binary"]
53     ]
54 })
55 feature_list.to_csv(PROCESSED_DIR / "feature_columns.csv",
    index=False)
```

Code 5.1: Main preprocessing logic from prepare_dataset.py

A separate script, `prepare_custom_dataset.py`, was later used for the additional custom normal and attack traffic. It followed the same general idea of column alignment, numeric conversion, binary labelling, and saving a processed custom dataset. The prepared data was later used by scikit-learn models during the training stage [27].

5.10 Discussion

The preprocessing stage showed that dataset preparation is one of the most important parts of a machine learning based IDS. Even with a well-known public dataset, several practical issues had to be handled, including encoding differences, invalid values, duplicate rows, and feature name consistency.

This work was necessary because a strong model cannot produce reliable results if the input data is not prepared correctly.

5.11 Summary

This chapter described how CICIDS2017 was prepared for the final IDS. The selected CSV files were merged, cleaned, converted into binary labels, and aligned with the live CICFlowMeter format. After preprocessing, the final dataset contained 2,810,958 rows and 79 usable features. This dataset was then used for the model training experiments in the next chapter.

Chapter 6

Model Training and Evaluation

6.1 Introduction

After the dataset was cleaned and prepared, the next step was to train and evaluate the machine learning models. The aim was to compare several models and choose the most suitable one for the intrusion detection task.

Four models were tested in the final supervised setup: Decision Tree, Random Forest, Support Vector Machine, and Neural Network. The task was binary classification, where normal traffic was labelled as 0 and attack traffic was labelled as 1.

6.2 Training Goal

The goal of this stage was not only to train one classifier, but to compare different machine learning approaches under the same conditions. Machine learning methods are commonly used in IDS research because they can learn patterns from labelled network traffic data [3, 4].

All models were trained on the same prepared dataset and evaluated with the same metrics. This made the final model selection based on experimental results, not only on assumptions.

6.3 Prepared Dataset Used for Training

The input of the training script was the cleaned binary dataset created in the preprocessing stage. This dataset was based on CICIDS2017, which provides labelled benign and attack traffic for intrusion detection experiments [8, 9].

The final prepared dataset contained 2,810,958 rows, 81 columns, and 79 usable feature columns. The other two columns were the original label and the generated binary label.

Although the full dataset was available, the first model comparison was performed on a stratified sample of 200,000 rows. This was done for practical reasons, especially because SVM and Neural Network training can take more time on large datasets.

6.4 Sampling, Splitting, and Scaling

Stratified sampling was used so that the ratio of normal and attack traffic stayed close to the original dataset. This was important because class imbalance is common in intrusion detection datasets [4, 25]. After sampling, the data contained 160,941 normal rows and 39,059 attack rows.

The sampled data was split into 80% training data and 20% test data. This produced 160,000 training rows and 40,000 test rows. A stratified split was used again so that both sets kept a similar class distribution.

Before training, feature scaling was applied with `StandardScaler`. Scaling was especially useful for SVM and Neural Network. Tree-based models do not depend on scaling in the same way, but the same preprocessing pipeline was kept for all models for consistency. The fitted scaler was saved and later reused in the live detection pipeline. The training and evaluation were implemented with scikit-learn [27].

6.5 Class Imbalance Handling

The prepared dataset had more normal samples than attack samples. To reduce the effect of this imbalance, class weights were used during training. The approximate class weights were 0.62 for normal traffic and 2.56 for attack traffic. This means that

mistakes on attack samples were treated as more important than mistakes on normal samples.

6.6 Trained Models

The first model was a Decision Tree classifier. Decision Trees split data step by step based on feature values until a decision is reached [12, 13]. In this work, the tree depth and split parameters were limited to reduce overfitting.

The second model was Random Forest. It combines many decision trees and uses their combined prediction for classification [14]. It was expected to perform well because Random Forest is usually strong on structured tabular data.

The third model was Support Vector Machine. SVM tries to separate classes by finding a decision boundary between them [15]. A linear version was used to keep training manageable on the available machine.

The fourth model was a Neural Network. Neural networks are built from layers of connected units and can learn more complex relationships in data [16, 17]. In this thesis, a multilayer perceptron classifier was used.

6.7 Evaluation Metrics

The models were evaluated using accuracy, precision, recall, F1-score, and training time. Confusion matrices and classification reports were also generated. These are common evaluation tools in supervised machine learning experiments [22, 23].

Accuracy shows the overall ratio of correct predictions, but it can be misleading when the classes are imbalanced. Precision shows how many samples predicted as attack were really attacks. Recall shows how many real attacks were detected. F1-score combines precision and recall, so it was the most important metric for selecting the final model.

6.8 Training Results

Table 6.1 shows the final comparison of the four trained models.

Table 6.1: Comparison of trained models

Model	Accuracy	Precision	Recall	F1-score	Time (s)
Random Forest	0.998800	0.996673	0.997184	0.996929	6.05
Decision Tree	0.996925	0.986461	0.997952	0.992173	2.98
Neural Network	0.991950	0.976584	0.982335	0.979451	13.58
SVM	0.931825	0.755348	0.962750	0.846530	42.89

Random Forest achieved the best overall result. It had the highest F1-score, very high precision and recall, and a practical training time. Decision Tree was also strong and trained quickly, but Random Forest was slightly better overall. The Neural Network gave a good result, but it did not outperform the tree-based models. SVM had high recall, but its lower precision and longer training time made it less suitable for the final system.

Figure 6.1 shows the confusion matrix of the selected Random Forest model. It gives a clearer view of how many normal and attack samples were classified correctly or incorrectly.

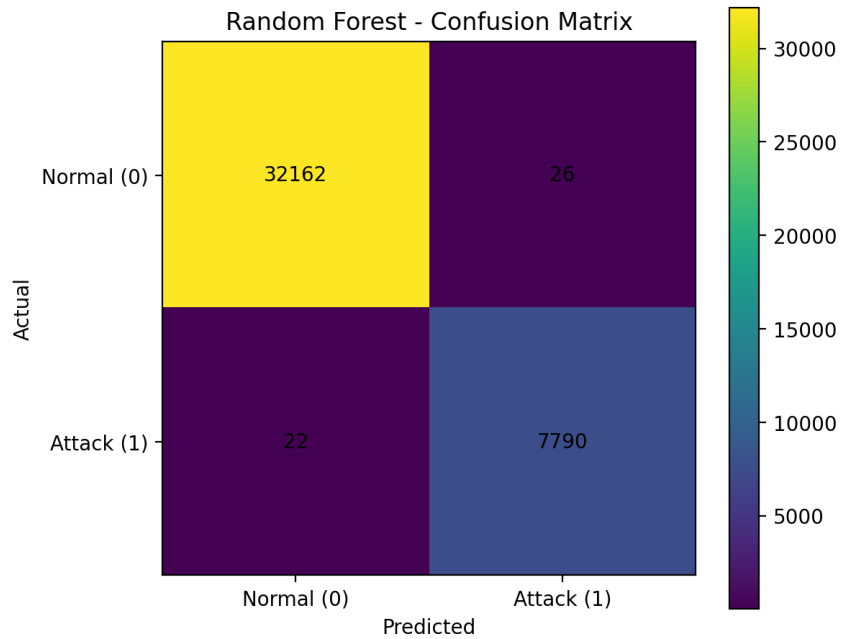


Figure 6.1: Confusion matrix of the selected Random Forest model.

6.9 Interpretation of the Results

The results show that Random Forest gave the best balance between precision, recall, and F1-score. This was important because an IDS should detect attacks, but

it should also avoid producing too many false alarms.

This result is also consistent with other intrusion detection research. IDS surveys report that tree-based and ensemble methods, including Random Forest, are often strong choices for network intrusion detection because they work well with structured traffic features and can model non-linear patterns [3, 4]. In this thesis, the same general conclusion was reached experimentally on the prepared CICIDS2017 flow-based data.

The comparison also showed that a more complex model is not always better. The Neural Network performed well, but it was still below Random Forest. SVM was less suitable in this experiment because it had lower precision and the longest training time.

6.10 Why Random Forest Was Selected

Random Forest was selected as the final model for the live IDS pipeline for three main reasons. First, it achieved the highest F1-score among all tested models. Second, it had strong precision and recall, which means it detected attacks well while keeping false positives low. Third, its training time was still practical.

Because of these results, Random Forest was used as the main classifier in the live detection stage.

6.11 Saved Training Outputs

The training process saved the fitted scaler, the trained models, the model comparison table, confusion matrix figures, classification reports, and the best model information file. These outputs were important because the live system needed the selected model and the same preprocessing configuration used during training.

6.12 Notes About Practical Implementation

The training was performed in Python. The main model training and evaluation were implemented with scikit-learn [27]. Pandas was used for reading and processing CSV files [26], and Matplotlib was used for saving figures such as confusion matrices [28].

The script was written so that the experiment could be repeated. The use of a stratified sample made the first comparison manageable while still keeping the class distribution realistic.

Code 6.1 shows the main training and comparison logic used in the model training script.

```
1 # Load processed binary dataset
2 df = pd.read_csv(DATA_FILE, low_memory=False)
3
4 feature_columns = [
5     col for col in df.columns
6     if col not in ["Label", "Label_Binary"]
7 ]
8
9 # Take stratified sample for manageable training
10 if USE_SAMPLE and len(df) > SAMPLE_SIZE:
11     _, df = train_test_split(
12         df,
13         test_size=SAMPLE_SIZE,
14         stratify=df["Label_Binary"],
15         random_state=RANDOM_STATE
16     )
17
18 X = df[feature_columns].astype(np.float32)
19 y = df["Label_Binary"].astype(int)
20
21 # Train/test split
22 X_train, X_test, y_train, y_test = train_test_split(
23     X, y,
24     test_size=TEST_SIZE,
25     stratify=y,
26     random_state=RANDOM_STATE
27 )
28
29 # Scale features
30 scaler = StandardScaler()
31 X_train_scaled = scaler.fit_transform(X_train)
```

```
32 X_test_scaled = scaler.transform(X_test)
33
34 # Train selected models
35 dt = DecisionTreeClassifier(
36     random_state=RANDOM_STATE,
37     class_weight="balanced",
38     max_depth=20
39 )
40 dt.fit(X_train_scaled, y_train)
41
42 rf = RandomForestClassifier(
43     n_estimators=150,
44     random_state=RANDOM_STATE,
45     n_jobs=-1,
46     class_weight="balanced_subsample",
47     max_depth=20
48 )
49 rf.fit(X_train_scaled, y_train)
50
51 svm_model = LinearSVC(
52     class_weight="balanced",
53     random_state=RANDOM_STATE,
54     max_iter=5000
55 )
56 svm_model.fit(X_train_scaled, y_train)
57
58 nn_model = MLPClassifier(
59     hidden_layer_sizes=(128, 64),
60     batch_size=1024,
61     max_iter=30,
62     early_stopping=True,
63     random_state=RANDOM_STATE
64 )
65 nn_model.fit(X_train_scaled, y_train)
66
67 # Save comparison results
```

```
68 results_df = pd.DataFrame(results)
69 results_df = results_df.sort_values(
70     by="F1_Score",
71     ascending=False
72 )
73 results_df.to_csv(
74     RESULTS_DIR / "model_comparison_results.csv",
75     index=False
76 )
```

Code 6.1: Main training and model comparison logic from train_ids.py

6.13 Possible Improvements

Several improvements could be explored in future work. The final Random Forest model could be trained on a larger part of the dataset, hyperparameter optimization could be added, and the task could be extended from binary classification to multiclass classification.

The selected model could also be tested on additional datasets such as UNSW-NB15, CSE-CIC-IDS2018, or CIC-DDoS2019. This would help evaluate how well the model generalizes outside the CICIDS2017 distribution [19, 20, 21].

6.14 Summary

This chapter described the training and evaluation of the four machine learning models. The models were trained on a stratified sample of the cleaned CICIDS2017 binary dataset and compared using accuracy, precision, recall, F1-score, and training time.

Random Forest achieved the best overall performance and was selected as the final model for the live IDS pipeline. The next chapter presents the implementation and validation of the live detection system.

Chapter 7

Live Implementation and Validation

7.1 Introduction

After the offline model comparison, the next step was to test whether the selected model could work in a live environment. The aim was to move from static dataset evaluation to a practical workflow that could process newly captured traffic and classify it automatically.

In the final system, Random Forest was used as the main classifier. The live setup captured traffic on Ubuntu, transferred the PCAP files to Windows through a shared folder, converted them into flow-based CSV files using CICFlowMeter, and classified the generated flows with the trained model [14, 11].

7.2 Live Pipeline

The live IDS pipeline had four main stages: traffic capture on Ubuntu, PCAP transfer through a VirtualBox shared folder, PCAP-to-CSV conversion with CICFlowMeter, and classification with the trained Random Forest model.

The Ubuntu capture process was automated with a Bash script. The script captured traffic from a selected interface for fixed time windows and saved each capture with a timestamped file name. Code 7.1 shows the capture script used in the live tests.

```
1 #!/bin/bash
2
3 INTERFACE="enp0s8"
```

```
4 SHARED_FOLDER="/media/sf_LiveFeed"
5 DURATION=30
6
7 echo "Starting live capture sensor..."
8 echo "Interface      : $INTERFACE"
9 echo "Shared folder  : $SHARED_FOLDER"
10 echo "Duration       : $DURATION seconds per file"
11 echo "Press Ctrl+C to stop."
12
13 while true; do
14     TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
15     TEMP_FILE="/tmp/live_capture_${TIMESTAMP}.pcap"
16     FINAL_FILE="${SHARED_FOLDER}/live_capture_${TIMESTAMP}.
17         pcap"
18
19     echo ""
20     echo "Capturing: $FINAL_FILE"
21
22     sudo timeout "$DURATION" tcpdump -i "$INTERFACE" -w "
23         $TEMP_FILE" -q
24
25     if [ -f "$TEMP_FILE" ]; then
26         sudo mv "$TEMP_FILE" "$FINAL_FILE"
27         sudo chmod 666 "$FINAL_FILE"
28         echo "[+] Saved: $FINAL_FILE"
29     else
30         echo "[-] Capture failed."
31     fi
32
33     sleep 1
34 done
```

Code 7.1: Ubuntu traffic capture script used in the live pipeline

Figure 7.1 shows the capture script while it was running on Ubuntu.

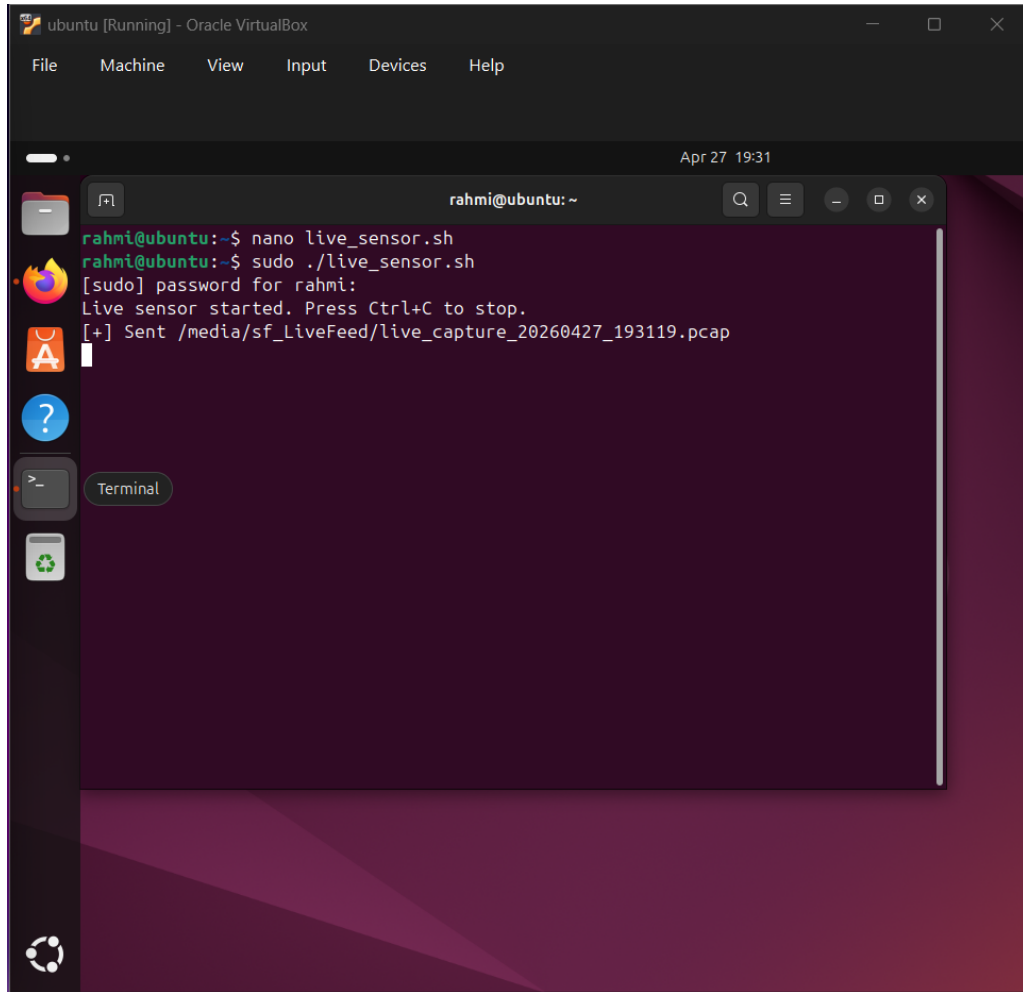


Figure 7.1: Ubuntu traffic capture script during execution.

After a PCAP file was created, the Windows live pipeline script detected it in the LiveFeed folder. CICFlowMeter then converted it into a flow-based CSV file. Figure 7.2 shows the conversion stage.

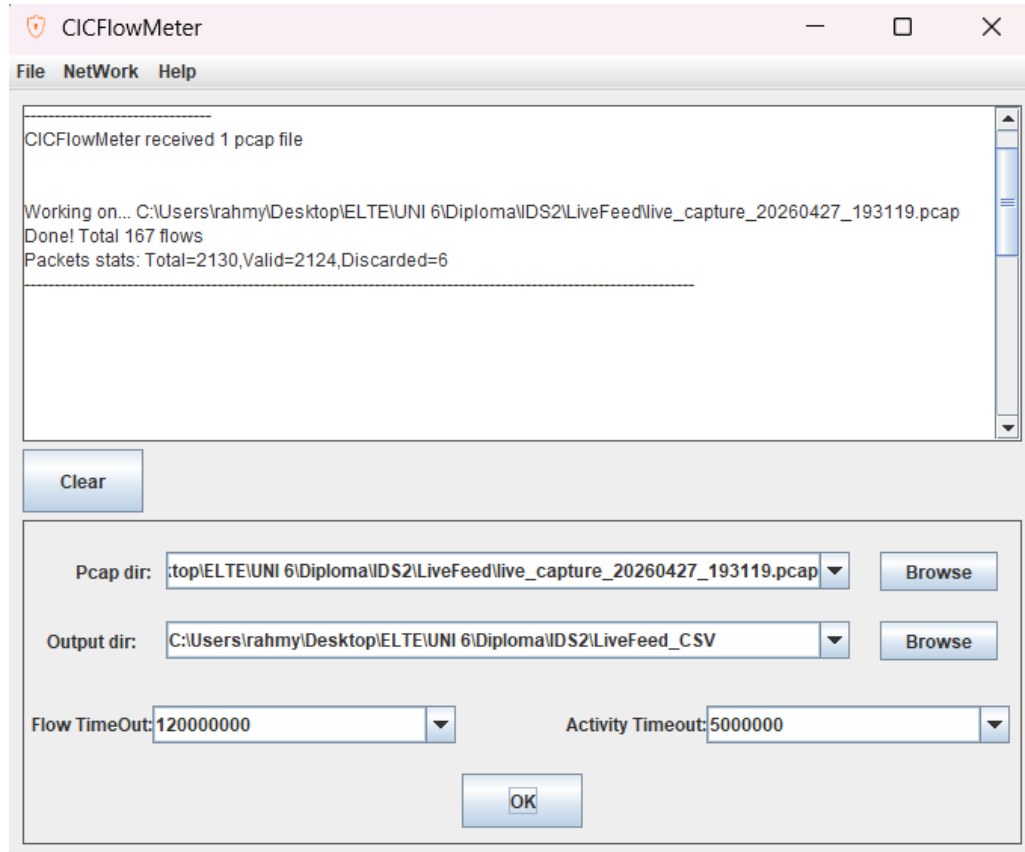


Figure 7.2: CICFlowMeter converting captured PCAP files into flow-based CSV files.

After conversion, the CSV file was prepared with the same feature list and scaler used during training. The model first classified each flow separately. Then the script counted the total flows, attack flows, and attack ratio, and produced a final decision as either NORMAL or ATTACK.

7.3 Validation Strategy

The validation was done in three parts. First, the model was tested on known CICIDS2017 CSV files. Second, the full live pipeline was tested on public CICIDS2017 PCAP files. Third, the system was tested on traffic captured in the local Ubuntu–Kali virtual lab.

This separation made it easier to understand whether a problem came from the model itself or from the live PCAP-to-CSV pipeline.

7.4 Validation on Public CICIDS2017 CSV Files

The first validation step used known CICIDS2017 CSV files. These tests checked whether the trained model behaved correctly on known benign and attack traffic [8, 9].

Table 7.1: Validation results on selected CICIDS2017 CSV files

File	Total	Attack	Ratio	Decision
Monday	529918	217	0.0004	NORMAL
Tuesday	445909	13921	0.0312	ATTACK
Wednesday	692703	252515	0.3645	ATTACK
Thu. WebAttacks	458968	2187	0.0048	NORMAL
Thu. Infiltration	288602	1134	0.0039	NORMAL
Friday Morning	191033	1510	0.0079	NORMAL
Friday PortScan	286467	159001	0.5550	ATTACK
Friday DDoS	225745	127914	0.5666	ATTACK

The results show that the model detected strong flow-based attacks such as PortScan and DDoS clearly. Lower-ratio or more mixed scenarios, such as WebAttacks and Infiltration, remained NORMAL at the file level.

7.5 Validation of the Full PCAP Pipeline

The next test used public CICIDS2017 PCAP files. This was important because it tested the complete chain from PCAP input to final prediction.

Table 7.2: Full PCAP pipeline validation

PCAP file	Total	Attack	Ratio	Decision
Friday public PCAP	695256	158689	0.2282	ATTACK
Monday public PCAP	453650	320	0.0007	NORMAL

The Friday PCAP was classified as ATTACK, while the Monday PCAP was classified as NORMAL. This confirmed that the full PCAP-to-CSV-to-model pipeline worked correctly on public test files.

7.6 Live Testing in the Virtual Lab

After the public validation, the system was tested on traffic captured in the local virtual lab. The lab used Ubuntu as the monitored machine, Kali Linux for controlled

attack traffic, and Windows as the processing machine. VirtualBox network modes such as host-only, NAT, and bridged networking were used during different tests [10].

During normal traffic tests, the live pipeline processed several captures and classified them as NORMAL. This was important because a useful IDS should not produce constant false alarms during ordinary traffic.

Some attack traffic generated in the local lab was initially classified as NORMAL by the original model. After comparing the live lab traffic with CICIDS2017 PortScan data, it became clear that the live traffic had different statistical properties from the public training data. This showed the practical problem of training-data mismatch in machine learning based intrusion detection [5].

7.7 Custom Attack Traffic Generation

To create custom attack traffic for the improved model, several controlled attack scenarios were generated inside the private Kali-Ubuntu virtual lab. The target machine was Ubuntu, and the attacking machine was Kali Linux. These tests were performed only inside the isolated lab network and were used only to collect traffic for IDS validation and model improvement.

Before starting the attack traffic collection, the Ubuntu target machine was prepared by enabling the services that would be attacked. Code 7.2 shows the commands used on Ubuntu.

```
1 python3 -m http.server 8000
2
3 sudo systemctl start ssh
```

Code 7.2: Target service preparation on Ubuntu

The first command started a lightweight HTTP server on port 8000. This created a simple web service that could be used for HTTP-based denial-of-service testing. The second command started the SSH service on port 22, which allowed failed login and brute-force style traffic to be generated during the experiment.

7.7.1 Port Scanning Traffic

The first type of custom attack traffic was port scanning. The purpose was to simulate reconnaissance behavior, where an attacker tries to discover open ports and services on a target machine.

Code 7.3 shows the two port scanning scenarios used in the lab.

```
1 sudo masscan -p 1-65535 192.168.56.102 --rate=5000
2
3 sudo nmap -sS -p- -T5 --max-retries 0 192.168.56.102
```

Code 7.3: Port scanning commands used in the virtual lab

The first command was used for the `attack_scan_01` capture. It used `masscan` to scan all TCP ports from 1 to 65535. The `-rate=5000` option limited the scan rate to 5000 packets per second. This created a fast and dense group of short network flows.

The second command was used for the `attack_scan_02` capture. It used `nmap` with a TCP SYN scan. The `-sS` option performs a half-open SYN scan, `-p-` scans all ports, `-T5` increases the scan speed, and `--max-retries 0` avoids repeated retries. This produced another type of scanning traffic for the IDS to learn from.

7.7.2 Denial-of-Service Traffic

The second type of custom attack traffic was denial-of-service traffic. These scenarios were used to create abnormal flow patterns related to resource exhaustion and high traffic volume.

Code 7.4 shows the denial-of-service commands used in the lab.

```
1 slowhttptest -c 1000 -H -g -o my_header_stats -i 10 -r 200 -u
   http://192.168.56.102:8000 -x 24 -p 3
2
3 sudo hping3 --udp --flood -d 1500 -p 53 192.168.56.102
```

Code 7.4: Denial-of-service traffic generation commands

The first command was used for the `attack_dos_01` capture. It used `slowhttptest` to generate slow HTTP traffic against the web server running on port 8000. The `-c 1000` option created up to 1000 connections, `-H` selected the

Slowloris-style header mode, `-i 10` controlled the interval between data chunks, and `-r 200` set the connection rate. This created application-layer DoS traffic.

The second command was used for the `attack_dos_02` capture. It used `hping3` to generate UDP flood traffic. The `-udp` option selected UDP traffic, `-flood` sent packets as fast as possible, `-d 1500` set the payload size, and `-p 53` targeted port 53. This created high-volume UDP traffic toward the Ubuntu target.

7.7.3 Failed Access Traffic

The third type of custom attack traffic was failed access traffic. These scenarios simulated repeated failed SSH login attempts against the Ubuntu target.

Code 7.5 shows the SSH brute-force style commands used in the lab.

```
1 hydra -l admin -P /usr/share/wordlists/fasttrack.txt ssh  
   ://192.168.56.102 -t 4  
2  
3 hydra -l root -P /usr/share/wordlists/fasttrack.txt ssh  
   ://192.168.56.102 -t 8
```

Code 7.5: Failed SSH access traffic generation commands

The first command was used for the `attack_failed_access_01` capture. It used `hydra` to attempt SSH logins with the username `admin`. The `-P` option selected the password list, and `-t 4` limited the process to 4 parallel tasks. This produced a moderate stream of failed SSH login attempts.

The second command was used for the `attack_failed_access_02` capture. It targeted the username `root` and used 8 parallel tasks with `-t 8`. This created a denser pattern of failed login traffic.

After each scenario was executed, the Ubuntu capture script saved the traffic as PCAP files. These PCAP files were then converted into flow-based CSV files using CICFlowMeter. The generated custom attack data was later combined with custom normal traffic and used to train the improved Random Forest model.

7.8 Improved Model with Custom Data

To reduce this mismatch, an improved Random Forest model was created. The original CICIDS2017 training data was combined with additional custom traffic

collected from the local test environment. The custom dataset included both normal traffic and controlled attack traffic from the Kali-Ubuntu lab.

Because the custom data was imbalanced, it was balanced before training. After balancing, the custom dataset contributed 1372 normal rows and 1372 attack rows. These rows were merged with a 200,000-row stratified sample of the CICIDS2017 binary dataset. The improved model was trained using Python and scikit-learn [27].

Table 7.3: Original and improved Random Forest results

Model	Acc.	Prec.	Recall	F1	Time (s)
Original RF	0.998800	0.996673	0.997184	0.996929	6.05
Improved RF	0.998964	0.996789	0.998021	0.997405	8.40

The improvement was small, but it showed that adding environment-specific traffic helped the model adapt better to the local setup.

7.9 Live Result of the Improved Model

The improved model was tested again in the full live pipeline. The first test was a normal capture, and the model classified all flows as normal. Two later attack captures were classified as ATTACK with very high attack ratios.

Table 7.4: Live detection results of the improved model

Capture	Total	Attack	Ratio	Decision
Normal live capture	242	0	0.0000	NORMAL
Attack live capture 1	11650	11647	0.9997	ATTACK
Attack live capture 2	67128	67104	0.9996	ATTACK

The original filenames were kept in the saved result files, but shorter names are used in the table to keep it readable.

Figure 7.3 shows an example terminal output from the live detection pipeline.

```

\results\live_predictions_improved\live_capture_20260427_143358.pcap_Flow_predictions.csv
Archived PCAP to: C:\Users\rahmy\Desktop\ELTE\UNI 6\Diploma\IDS2\IDS_NORMAL\data\live_pcap_archive\live_capture_20260427_143358.pcap

Converting PCAP with CICFlowMeter: live_capture_20260427_143428.pcap
Generated CSV file(s):
- live_capture_20260427_143428.pcap_Flow.csv

Classifying CSV: live_capture_20260427_143428.pcap_Flow.csv
Total flows : 82
Normal flows : 82
Attack flows : 0
Attack ratio : 0.0000
Decision : NORMAL
Saved : C:\Users\rahmy\Desktop\ELTE\UNI 6\Diploma\IDS2\IDS_NORMAL\results\live_predictions_improved\live_capture_20260427_143428.pcap_Flow_predictions.csv
Archived PCAP to: C:\Users\rahmy\Desktop\ELTE\UNI 6\Diploma\IDS2\IDS_NORMAL\data\live_pcap_archive\live_capture_20260427_143428.pcap

Converting PCAP with CICFlowMeter: live_capture_20260427_143459.pcap
Generated CSV file(s):
- live_capture_20260427_143459.pcap_Flow.csv

Classifying CSV: live_capture_20260427_143459.pcap_Flow.csv
Total flows : 132
Normal flows : 132
Attack flows : 0
Attack ratio : 0.0000
Decision : NORMAL
Saved : C:\Users\rahmy\Desktop\ELTE\UNI 6\Diploma\IDS2\IDS_NORMAL\results\live_predictions_improved\live_capture_20260427_143459.pcap_Flow_predictions.csv
Archived PCAP to: C:\Users\rahmy\Desktop\ELTE\UNI 6\Diploma\IDS2\IDS_NORMAL\data\live_pcap_archive\live_capture_20260427_143459.pcap

```

Figure 7.3: Example output of the live detection pipeline showing the final decision.

These results were important because they tested both sides of the improved system. The normal capture was not falsely detected as an attack, while the attack captures were detected clearly.

7.10 Limitations

The live system still has limitations. It works in repeated capture windows instead of a fully continuous real-time stream. The lab traffic is also simpler than traffic in a real production network. In addition, the final output is binary, so it only reports NORMAL or ATTACK and does not identify the exact attack category.

7.11 Summary

This chapter presented the live implementation and validation of the IDS. The system used Ubuntu for traffic capture, a shared folder for transfer, CICFlowMeter for flow extraction, and Random Forest for classification.

The results showed that the model worked on known CICIDS2017 files, that the full PCAP pipeline worked correctly on public PCAP files, and that the improved model trained with custom data detected live attack traffic in the local Kali–Ubuntu lab.

Chapter 8

Conclusion and Future Work

8.1 Introduction

This chapter summarizes the main results of the thesis and presents possible future improvements. The project started with an anomaly detection idea, but during implementation it was redesigned into a supervised flow-based IDS. Because of this, the conclusion is not only about model accuracy, but also about the practical design decisions and lessons learned.

8.2 Summary of the Thesis

The main goal of the thesis was to design and implement a machine learning based intrusion detection system that can classify network traffic as normal or attack.

The first idea was based on normal-only anomaly detection. In that version, Ubuntu was used for traffic capture, Google Drive was used for file transfer, TShark was used for PCAP-to-CSV conversion, and TimesNet was selected as the model. This approach did not work reliably because the generated CSV files did not provide a stable flow-based feature representation, and the training data did not match the testing data well enough.

After this, the project was redesigned. The final version used supervised learning with the CICIDS2017 dataset. The data was cleaned, merged, and converted into a binary classification format where **BENIGN** traffic was normal and all other labels were attack traffic.

Four models were trained and compared: Decision Tree, Random Forest, Support Vector Machine, and Neural Network. Random Forest achieved the best overall result and was selected for the live IDS pipeline.

The final system used Ubuntu for traffic capture, Kali Linux for controlled abnormal traffic generation, and Windows for processing. CICFlowMeter converted PCAP files into flow-based CSV files, and the trained Random Forest model classified the generated flows.

8.3 Main Results

The offline model comparison showed that Random Forest performed best among the tested models. It achieved the highest F1-score and also had strong accuracy, precision, and recall. Decision Tree was close, Neural Network performed well but stayed below the tree-based models, and SVM had the weakest result in this experiment.

The live pipeline was also successfully implemented. It processed PCAP files, converted them into CSV files, applied the trained model, and produced a final NORMAL or ATTACK decision.

The validation showed that known CICIDS2017 PortScan and DDoS traffic were detected as ATTACK, while the mostly benign Monday file was detected as NORMAL. The full PCAP pipeline also classified public Friday traffic as ATTACK and public Monday traffic as NORMAL. In the local virtual lab, normal live traffic was classified as NORMAL, and after adding custom data, live attack captures were detected clearly as ATTACK.

8.4 Research Questions Revisited

The main practical question was whether it is possible to build a machine learning based IDS that works not only on public dataset files, but also in a live testing pipeline. Based on the final system, the answer is yes. However, the project also showed that a working IDS needs more than a trained model. Stable feature extraction, consistent preprocessing, and a practical file transfer workflow were also necessary.

Another question was whether the normal-only anomaly detection approach was suitable for this project. In this work, it was not suitable as the final solution. The idea was interesting, but the feature extraction and live data pipeline were not stable enough. Under these conditions, supervised learning with a labelled public dataset was a more practical choice.

8.5 Main Lessons Learned

The most important lesson was that feature consistency is critical. If the training data and live data do not have the same structure, the model cannot be trusted.

The second lesson was that a complete IDS is a pipeline, not only a machine learning model. Capture, transfer, conversion, preprocessing, prediction, and result storage all affect the final system.

The project also showed that simpler models can perform very well on structured flow-based data. In this thesis, Random Forest performed better than the Neural Network and SVM.

Finally, the first failed approach was still useful. It helped identify the problems that had to be solved in the final design.

8.6 Strengths and Limitations

The main strength of the final system is that it connects offline dataset-based training with a working live detection pipeline. It uses a public dataset, compares several models, selects the final model based on measured results, and validates the system with both public files and live captured traffic.

The system also has limitations. The final output is binary, so it only reports NORMAL or ATTACK and does not identify the exact attack type. The live pipeline is file-based and window-based, not a fully continuous high-speed IDS. The live validation was done in a virtual lab, which is useful for controlled testing but simpler than a real enterprise network. Also, the first model comparison used a stratified sample instead of training all models on the full dataset, mainly for practical reasons.

8.7 Future Work

Future work could extend the system from binary classification to multi-class classification, so that the model can identify the attack type. The final Random Forest model could also be trained on a larger part of the dataset or tuned with hyperparameter optimization.

Another useful direction would be testing the model on additional datasets, such as newer CIC datasets, UNSW-NB15, or CIC-DDoS2019. This would show how well the model generalizes outside CICIDS2017.

The live system could also be improved by replacing the current file-based processing with a more direct real-time architecture. A dashboard, alert system, and better result visualization would also make the prototype easier to use.

The anomaly detection idea could also be revisited in the future, but it would need a better flow-based data pipeline and a more stable normal traffic dataset.

8.8 Final Remarks

This thesis showed that building a machine learning based IDS is both a modelling task and a system design task. The final result depended on dataset preparation, feature extraction, model comparison, tool selection, and live pipeline design.

The project achieved its main goal. A working flow-based intrusion detection prototype was designed, implemented, tested, and validated in a virtual lab environment.

Acknowledgements

I would like to thank my supervisor for the guidance and support during the preparation of this thesis. I would also like to thank everyone who helped me during the implementation and testing of the project.

Bibliography

- [1] Karen Scarfone and Peter Mell. *Guide to Intrusion Detection and Prevention Systems (IDPS)*. Tech. rep. NIST Special Publication 800-94. National Institute of Standards and Technology, 2007. DOI: 10.6028/NIST.SP.800-94.
- [2] Dorothy E. Denning. “An Intrusion-Detection Model”. In: *IEEE Transactions on Software Engineering* SE-13.2 (1987), pp. 222–232. DOI: 10.1109/TSE.1987.232894.
- [3] Anna L. Buczak and Erhan Guven. “A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection”. In: *IEEE Communications Surveys & Tutorials* 18.2 (2016), pp. 1153–1176. DOI: 10.1109/COMST.2015.2494502.
- [4] Ansam Khraisat et al. “Survey of Intrusion Detection Systems: Techniques, Datasets and Challenges”. In: *Cybersecurity* 2.20 (2019). DOI: 10.1186/s42400-019-0038-7.
- [5] Robin Sommer and Vern Paxson. “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection”. In: *IEEE Symposium on Security and Privacy* (2010), pp. 305–316. DOI: 10.1109/SP.2010.25.
- [6] Wireshark Foundation. *tshark(1) Manual Page*. 2026. URL: <https://www.wireshark.org/docs/man-pages/tshark.html> (visited on 04/25/2026).
- [7] Haixu Wu et al. *TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis*. 2023. URL: <https://arxiv.org/abs/2210.02186> (visited on 04/25/2026).
- [8] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization”. In: *Proceedings of the 4th International Conference on*

- Information Systems Security and Privacy*. SciTePress, 2018, pp. 108–116. DOI: 10.5220/0006639801080116.
- [9] Canadian Institute for Cybersecurity. *Intrusion Detection Evaluation Dataset (CIC-IDS2017)*. 2017. URL: <https://www.unb.ca/cic/datasets/ids-2017.html> (visited on 04/25/2026).
- [10] Oracle. *Oracle VM VirtualBox User Manual: Virtual Networking*. 2026. URL: <https://www.virtualbox.org/manual/ch06.html> (visited on 04/25/2026).
- [11] Canadian Institute for Cybersecurity. *CICFlowMeter*. 2017. URL: <https://www.unb.ca/cic/research/applications.html> (visited on 04/25/2026).
- [12] Leo Breiman et al. *Classification and Regression Trees*. Wadsworth, 1984.
- [13] J. Ross Quinlan. *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1993.
- [14] Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32. DOI: 10.1023/A:1010933404324.
- [15] Corinna Cortes and Vladimir Vapnik. “Support-Vector Networks”. In: *Machine Learning* 20.3 (1995), pp. 273–297. DOI: 10.1007/BF00994018.
- [16] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning Representations by Back-Propagating Errors”. In: *Nature* 323 (1986), pp. 533–536. DOI: 10.1038/323533a0.
- [17] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [18] Mahbod Tavallaei et al. “A Detailed Analysis of the KDD CUP 99 Data Set”. In: *Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications* (2009), pp. 1–6. DOI: 10.1109/CISDA.2009.5356528.
- [19] Nour Moustafa and Jill Slay. “UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems”. In: *2015 Military Communications and Information Systems Conference*. 2015, pp. 1–6. DOI: 10.1109/MilCIS.2015.7348942.

- [20] Canadian Institute for Cybersecurity. *CSE-CIC-IDS2018 on AWS*. 2018. URL: <https://www.unb.ca/cic/datasets/ids-2018.html> (visited on 04/25/2026).
- [21] Canadian Institute for Cybersecurity. *CIC-DDoS2019 Dataset*. 2019. URL: <https://www.unb.ca/cic/datasets/ddos-2019.html> (visited on 04/25/2026).
- [22] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [23] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. 2nd ed. Springer, 2009.
- [24] Wireshark Foundation. *tcpdump: Capturing with tcpdump for Viewing with Wireshark*. 2026. URL: https://www.wireshark.org/docs/wsug_html_chunked/AppToolstcpdump.html (visited on 04/25/2026).
- [25] Markus Ring et al. “A Survey of Network-Based Intrusion Detection Data Sets”. In: *Computers & Security* 86 (2019), pp. 147–167. DOI: 10.1016/j.cose.2019.06.005.
- [26] Wes McKinney. “Data Structures for Statistical Computing in Python”. In: *Proceedings of the 9th Python in Science Conference*. 2010, pp. 56–61.
- [27] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [28] John D. Hunter. “Matplotlib: A 2D Graphics Environment”. In: *Computing in Science & Engineering* 9.3 (2007), pp. 90–95. DOI: 10.1109/MCSE.2007.55.

List of Figures

4.1	Architecture of the final IDS system.	20
4.2	Virtual lab network used for live IDS testing.	21
4.3	CICFlowMeter interface used for PCAP-to-CSV conversion.	22
6.1	Confusion matrix of the selected Random Forest model.	38
7.1	Ubuntu traffic capture script during execution.	45
7.2	CICFlowMeter converting captured PCAP files into flow-based CSV files.	46
7.3	Example output of the live detection pipeline showing the final decision.	52

List of Tables

6.1	Comparison of trained models	38
7.1	Validation results on selected CICIDS2017 CSV files	47
7.2	Full PCAP pipeline validation	47
7.3	Original and improved Random Forest results	51
7.4	Live detection results of the improved model	51

List of Codes

4.1	Main folder structure of the IDS project	24
4.2	Dockerfile used for the Python machine learning environment	26
4.3	Docker commands used to build and run the IDS training environment	26
5.1	Main preprocessing logic from prepare_dataset.py	31
6.1	Main training and model comparison logic from train_ids.py	40
7.1	Ubuntu traffic capture script used in the live pipeline	43
7.2	Target service preparation on Ubuntu	48
7.3	Port scanning commands used in the virtual lab	49
7.4	Denial-of-service traffic generation commands	49
7.5	Failed SSH access traffic generation commands	50